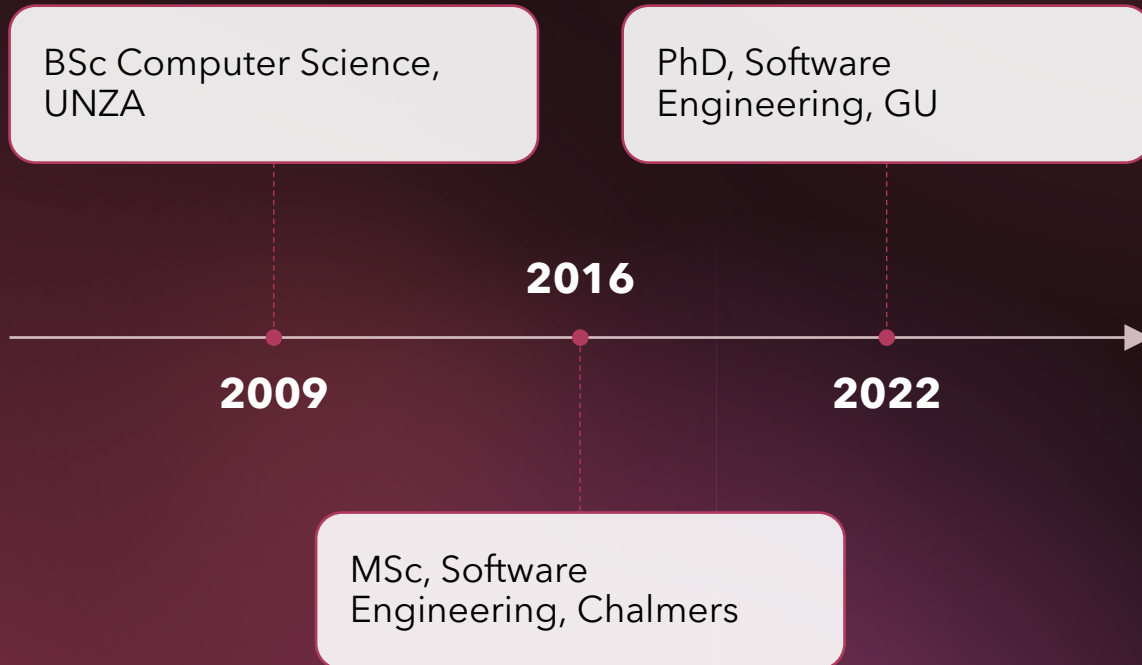




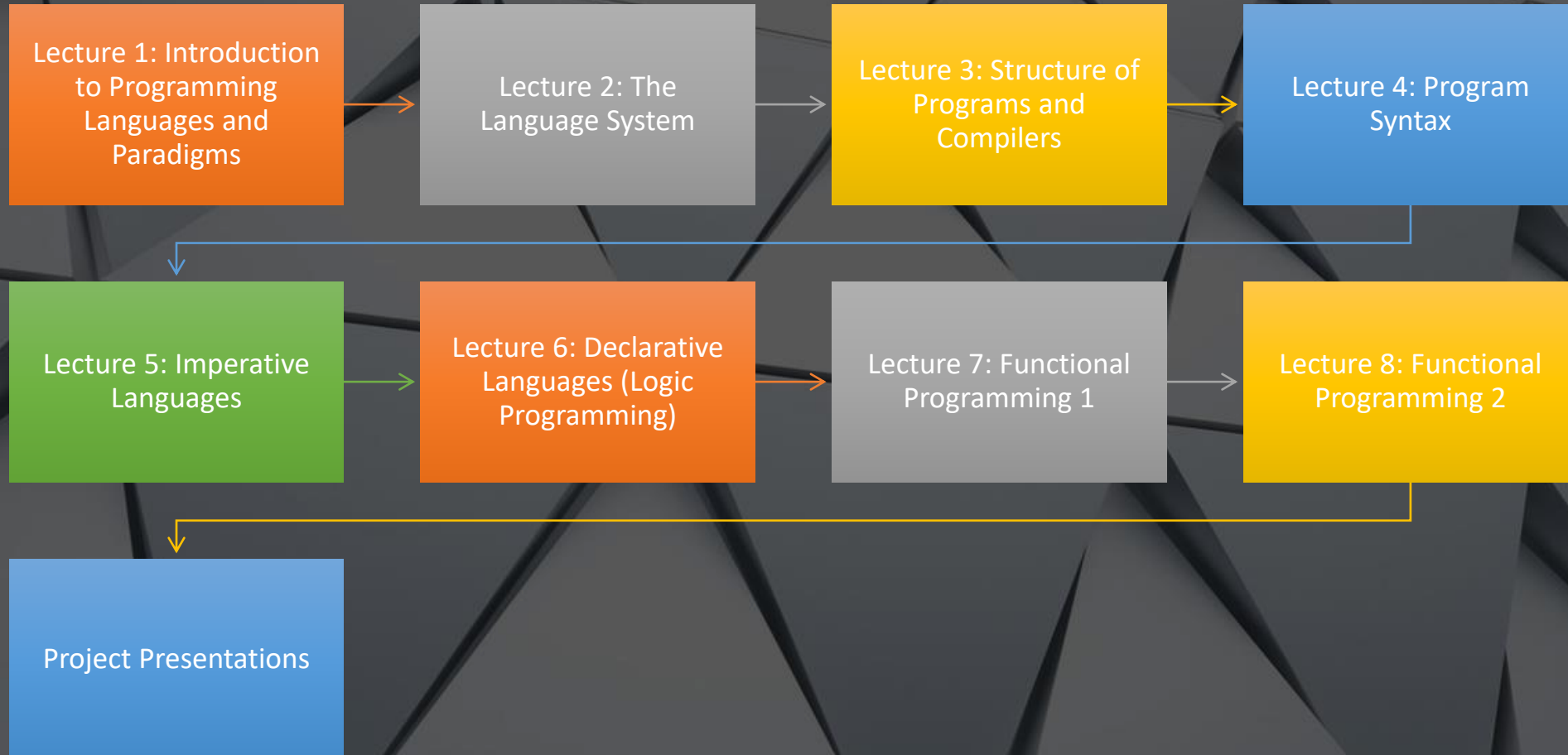
CSC 3301: Programming Language Paradigms

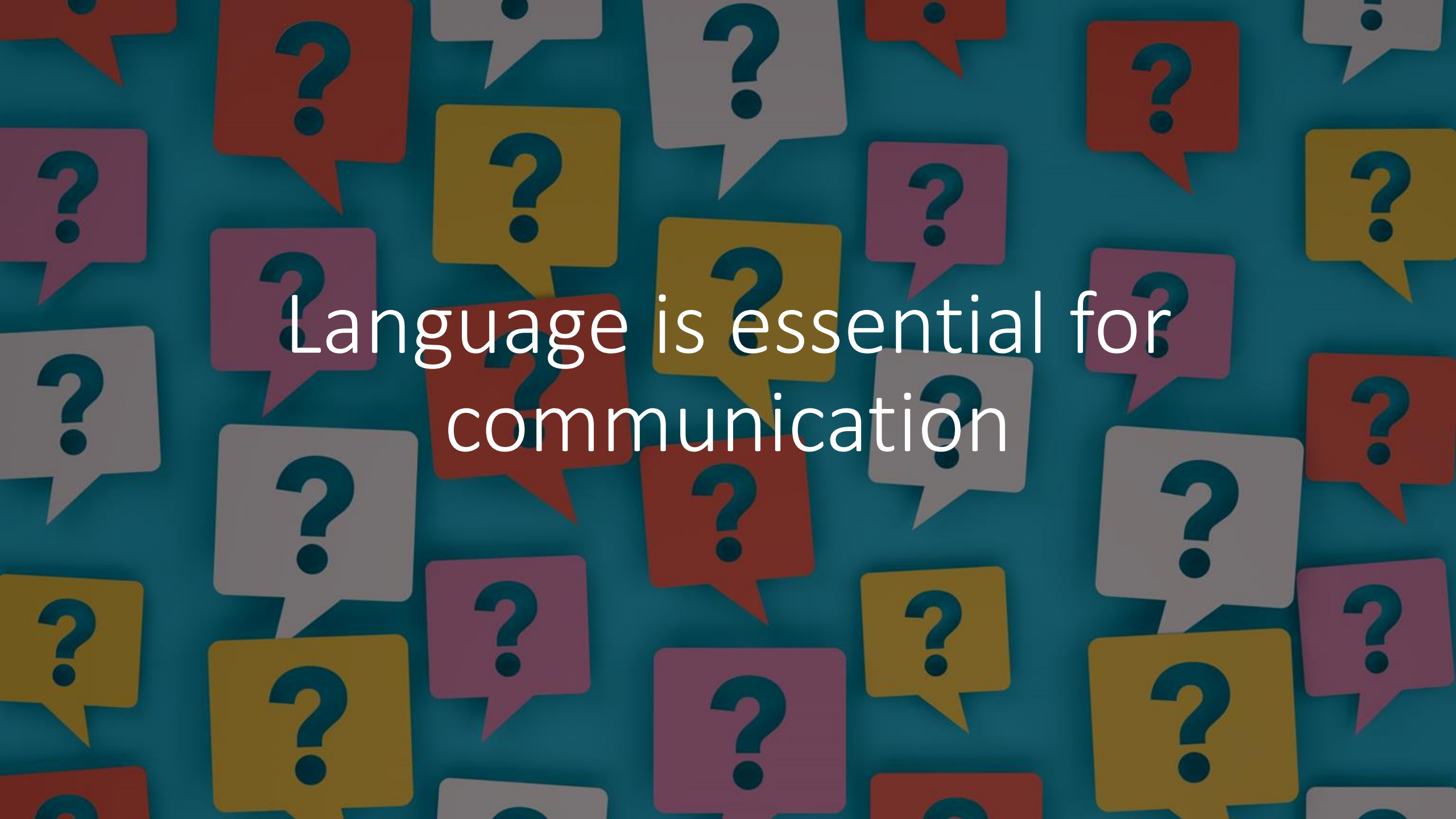
Dr Mukelabai Mukelabai

A bit about me



Course Outline



The background of the image is a dark teal color, densely populated with numerous speech bubbles. Each speech bubble is a different color, including shades of red, yellow, purple, and grey. Inside each speech bubble is a large, dark blue question mark. The speech bubbles are scattered across the entire frame, creating a pattern that suggests a continuous stream of questions or a state of inquiry.

Language is essential for
communication



A Computer functions through the interaction
of hardware and software

Software refers to
programs—sets of
instructions

```
for object to mirror  
mirror_mod.mirror_object =
```

```
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
selection at the end -add  
mirror_ob.select= 1  
mirror_ob.select=1  
context.scene.objects.update  
("Selected" + str(modifier))  
mirror_ob.select = 0  
= bpy.context.selected_objects  
data.objects[one.name].select
```

```
print("please select exactly  
-- OPERATOR CLASSES -----
```

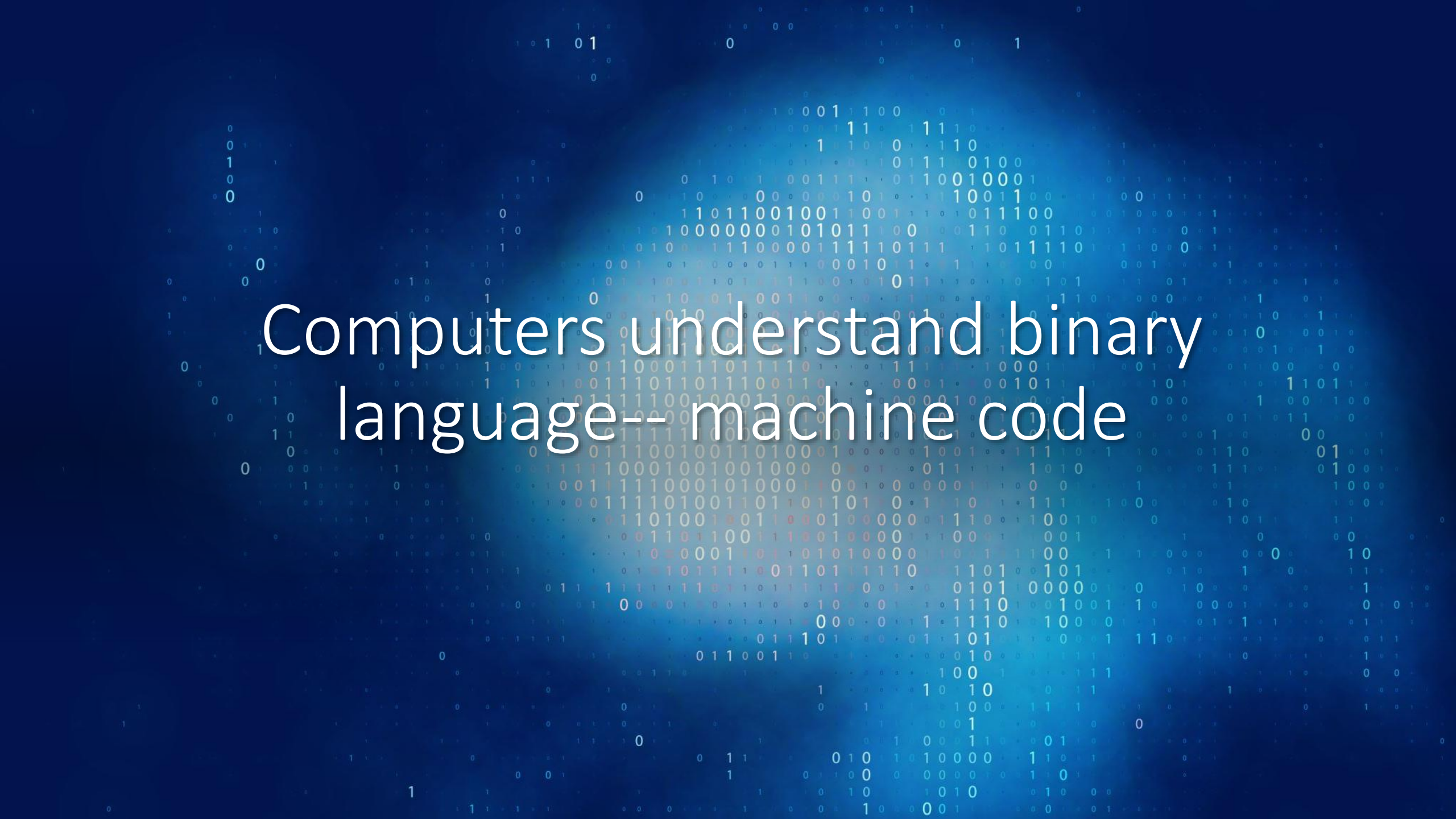
```
types.Operator):  
    X mirror to the selected  
    object.mirror_mirror_x"  
    mirror X"
```

Computer instructions must
be in a specific language



Language

- Gud är God
- Banana



Computers understand binary
language--machine code

Major categories

Machine code

Assembly language

High level languages



History

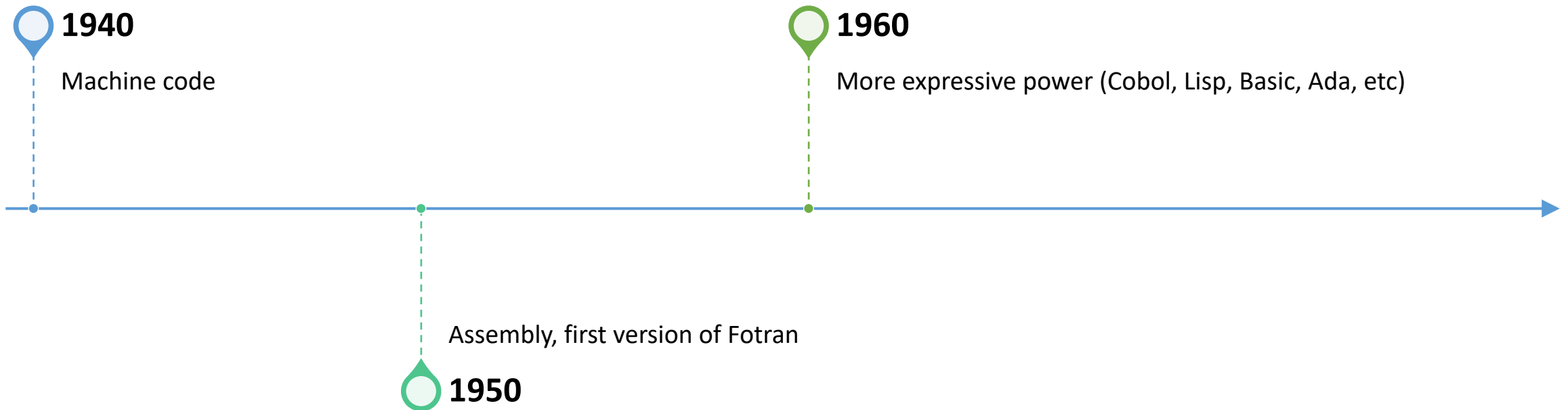


HIGHER LEVEL LANGUAGES ARE MORE
PROBLEM ORIENTED THAN MACHINE-
ORIENTED

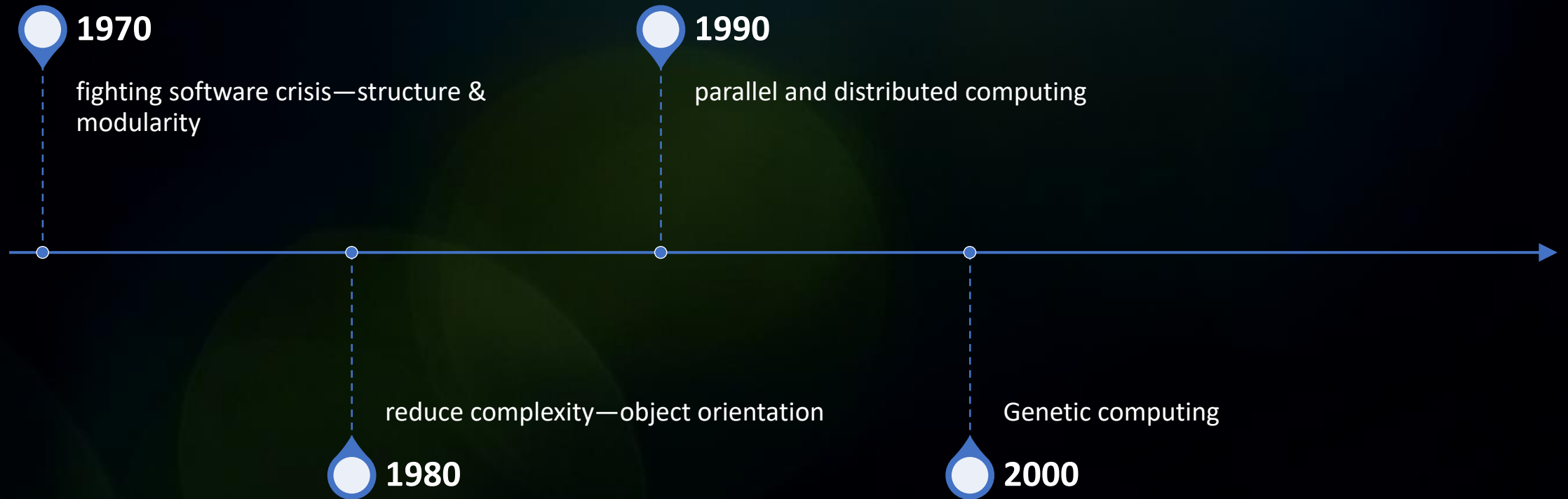


EARLY LANGUAGES INCLUDE ALGOL,
FOTRAN, PASCAL, BASIC, ADA, ETC

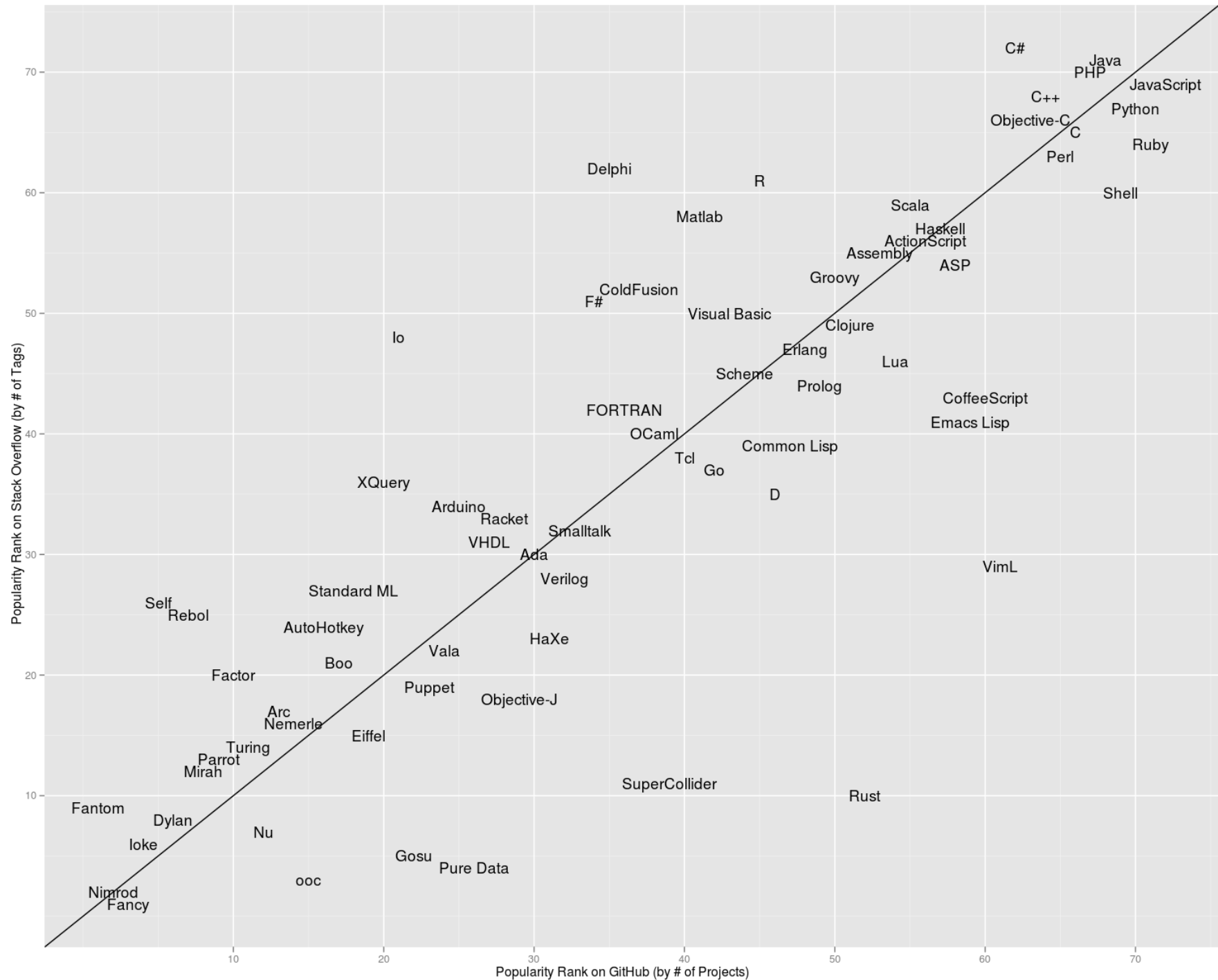
History

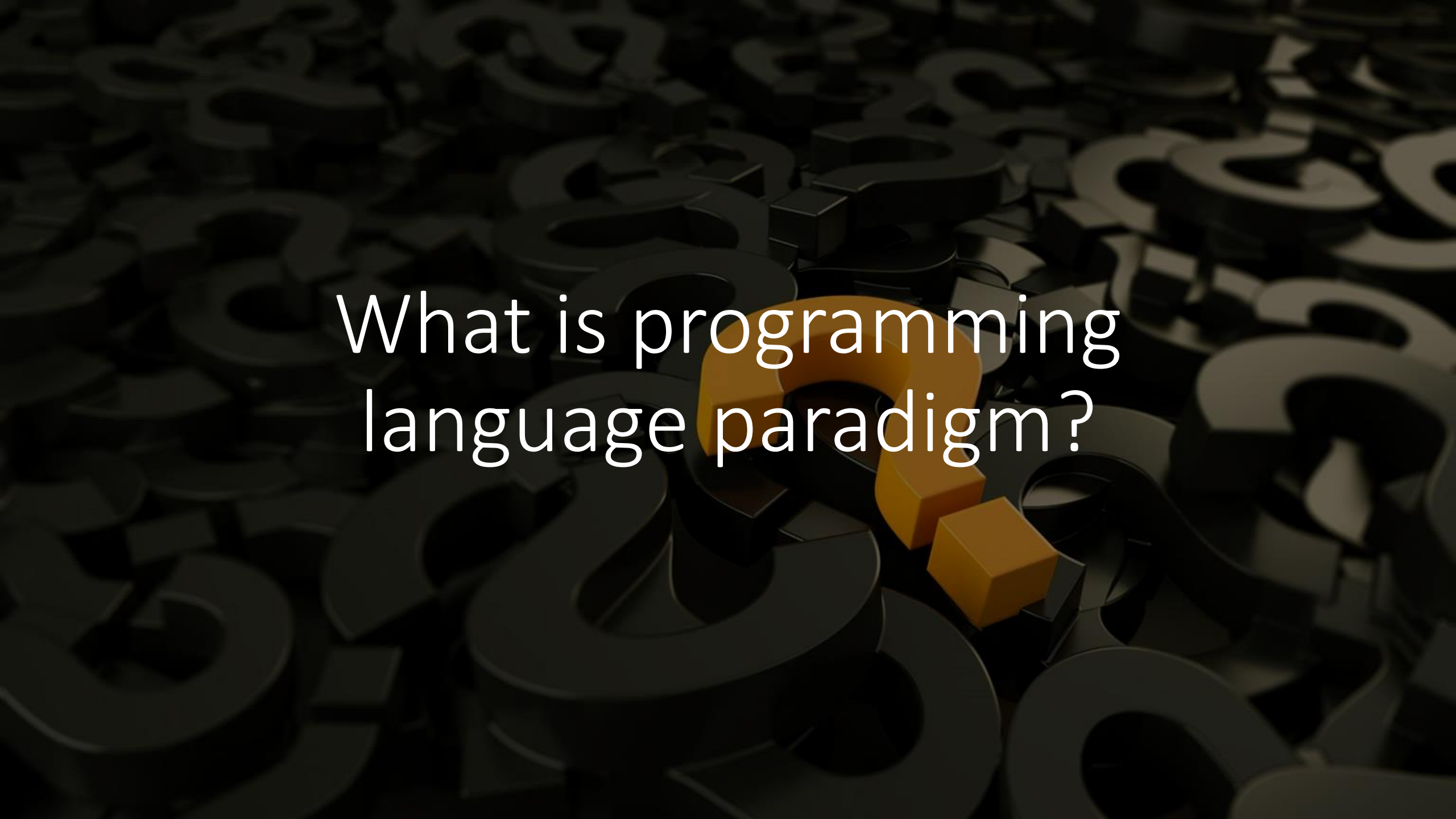


History

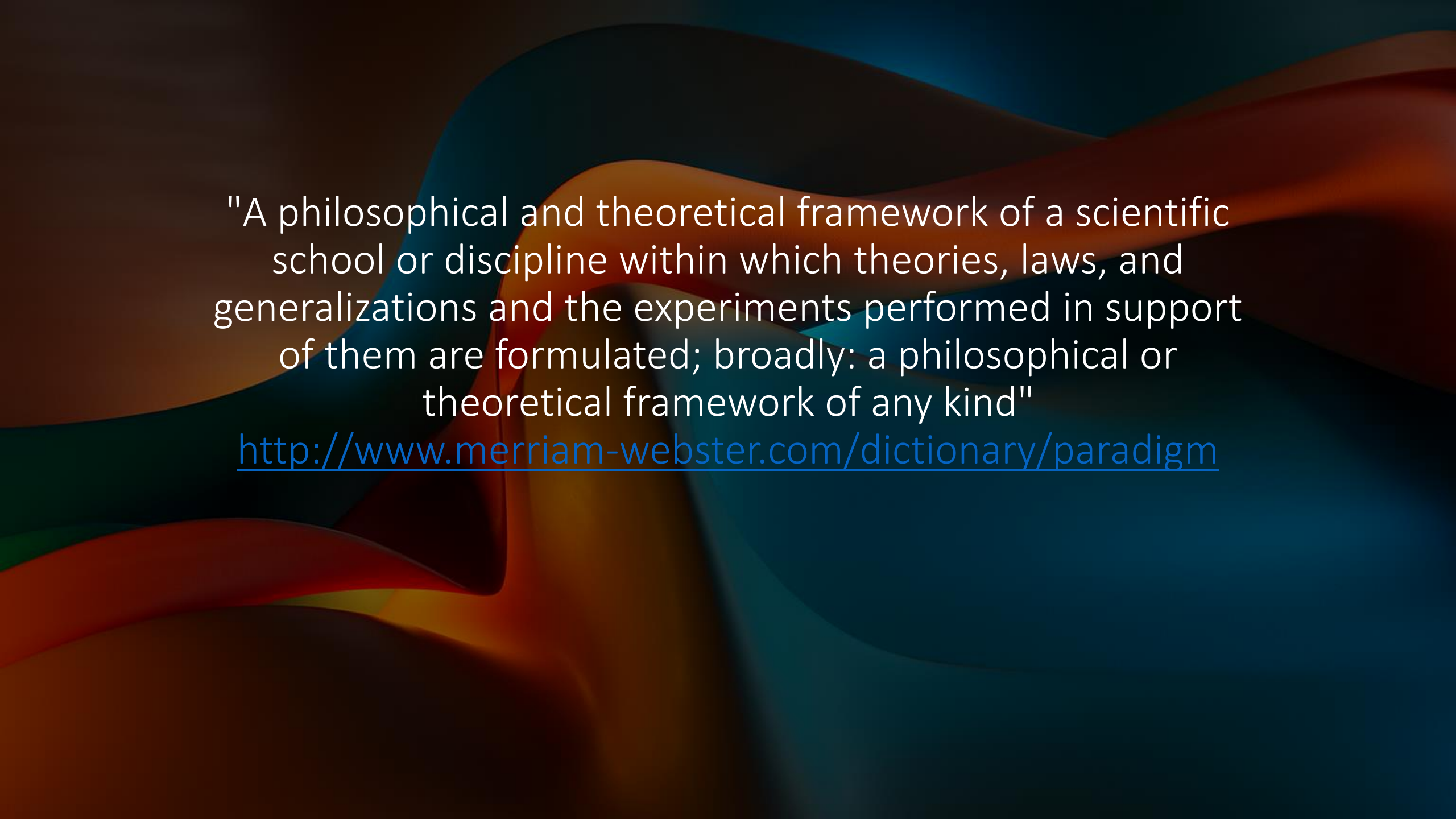


Different programming languages
have been introduced to reason
about problems in different ways



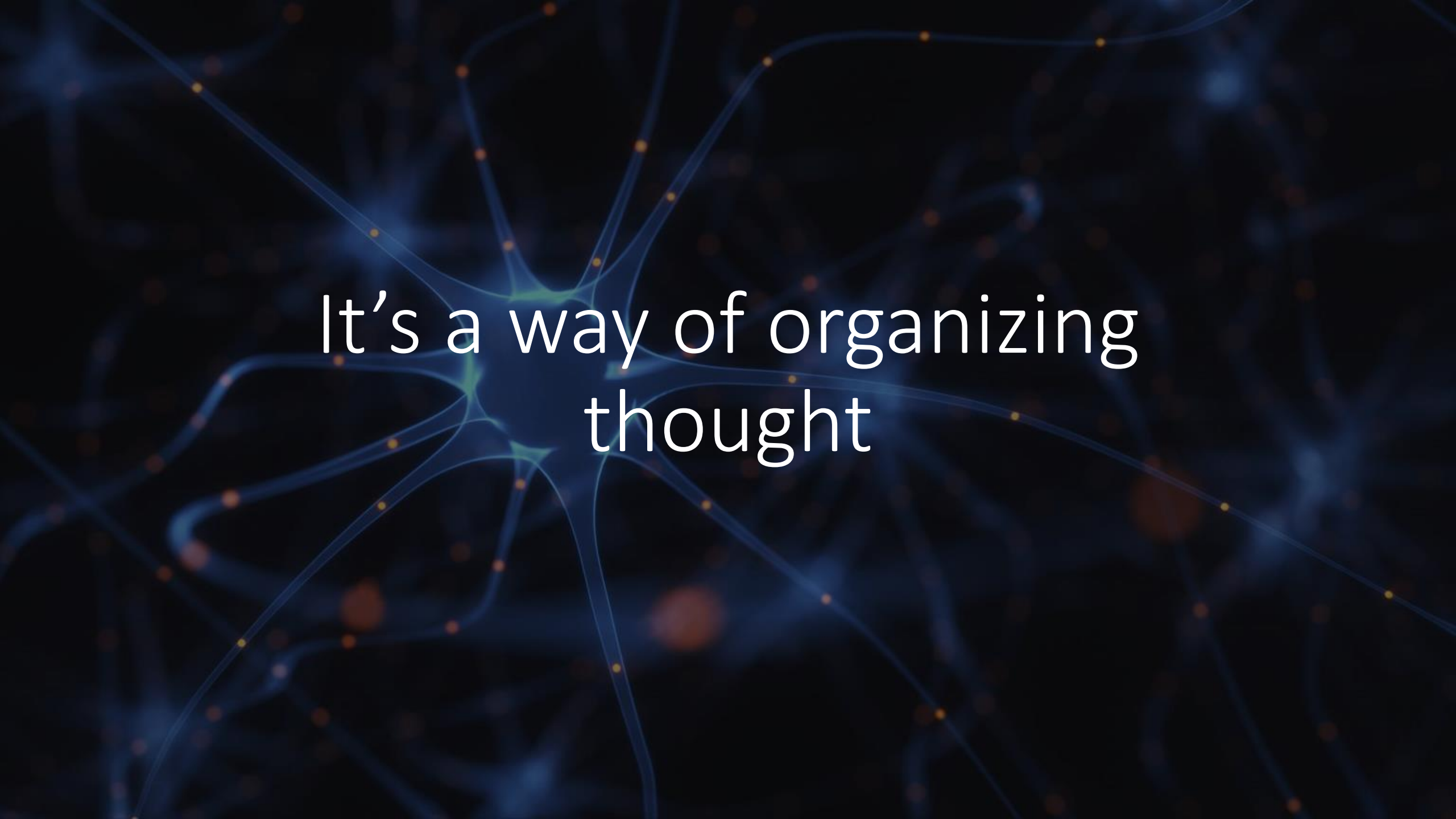


What is programming
language paradigm?



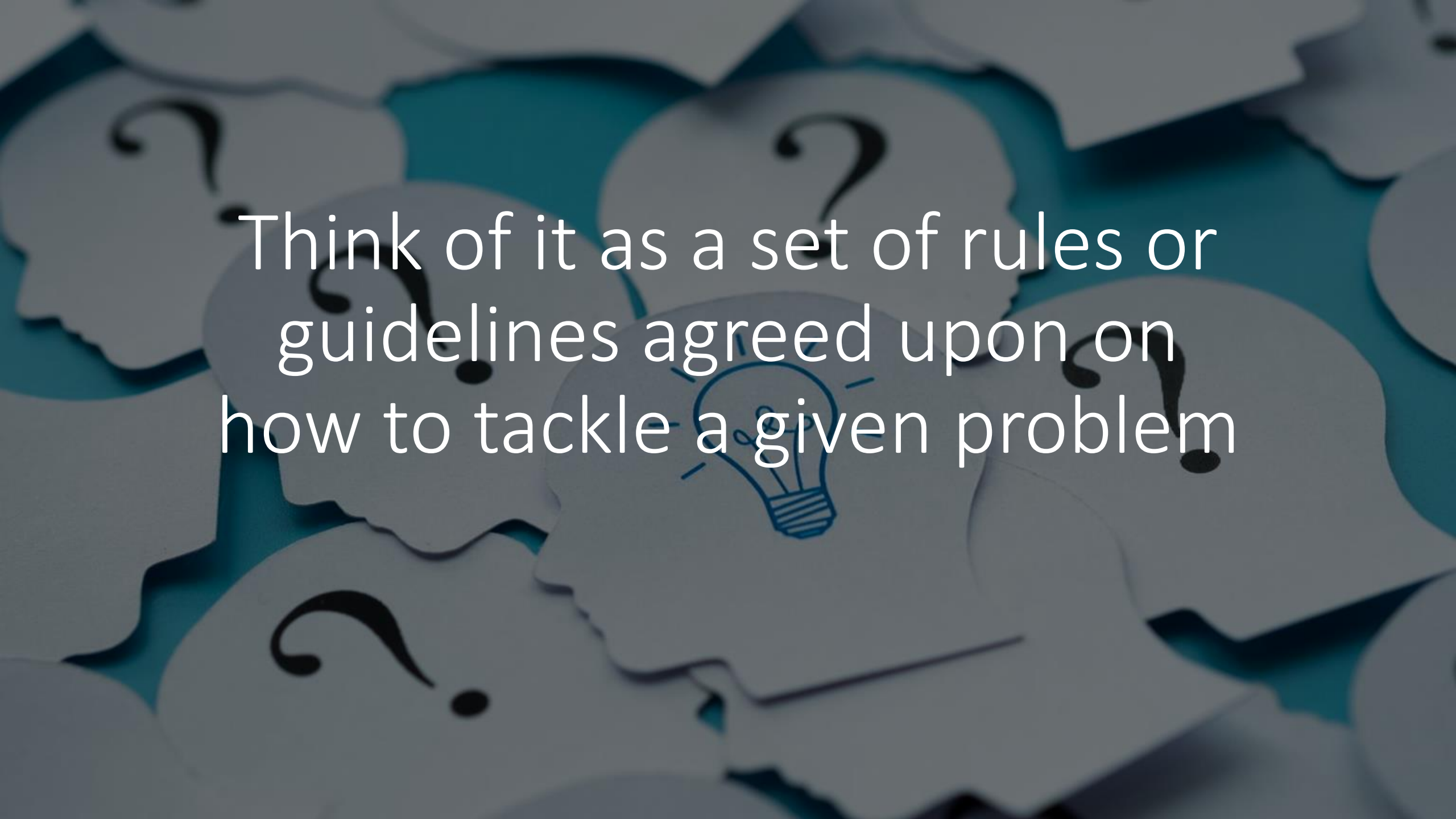
"A philosophical and theoretical framework of a scientific school or discipline within which theories, laws, and generalizations and the experiments performed in support of them are formulated; broadly: a philosophical or theoretical framework of any kind"

<http://www.merriam-webster.com/dictionary/paradigm>

The background is a dark blue field filled with a complex network of thin, glowing blue lines. These lines intersect and branch out, creating a web-like structure. Small, bright orange-yellow dots are scattered along these lines, acting as nodes or points of connection. The overall effect is reminiscent of a neural network or a complex data visualization.

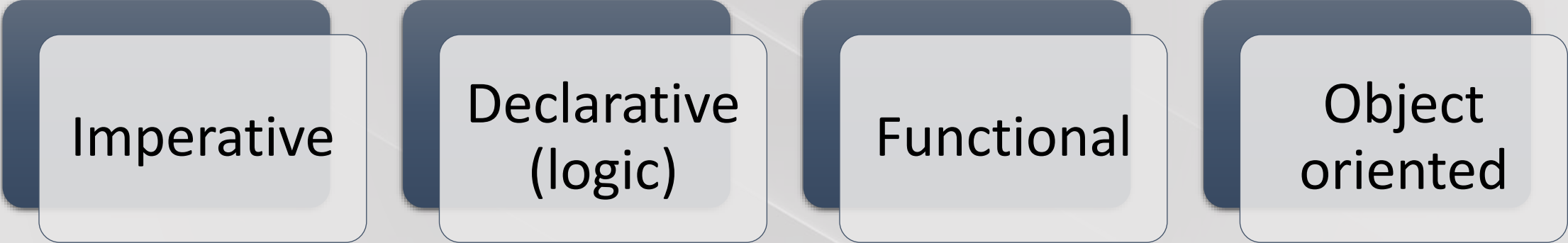
It's a way of organizing
thought

A programming language
paradigm is not a language;
neither is it a tool

The background features a collage of puzzle pieces in shades of grey and teal. Several pieces contain black question marks, and one central piece features a blue line-art lightbulb. The text is overlaid on this background.

Think of it as a set of rules or
guidelines agreed upon on
how to tackle a given problem

Four Main Language Paradigms



Imperative

Declarative
(logic)

Functional

Object
oriented



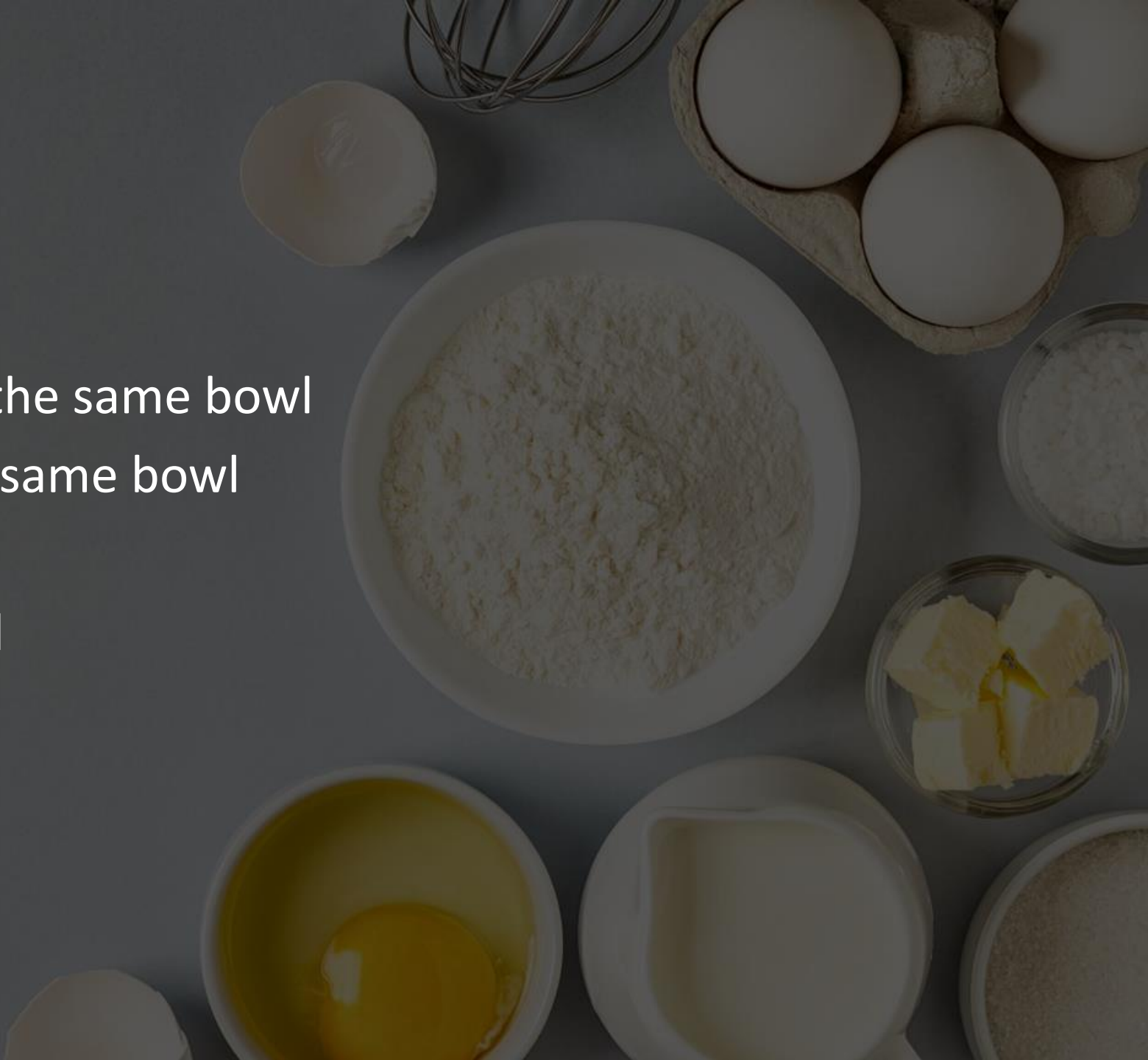
Imperative Paradigm

First do this and next do that

- Discipline
 - Hardware technology
- Incremental *change of the program state* as a function of *time*.
- Just like every day food recipes
- Typical commands offered by imperative languages
 - Assignment, IO, procedure calls
- Language representatives
 - Fortran, Algol, Pascal, Basic, C
- The natural abstraction is the procedure—abstract commands into one
 - "Procedural programming"

Example

- 1- Pour flour in a bowl
- 2- Pour a couple eggs in the same bowl
- 3- Pour some milk in the same bowl
- 4- Mix the ingredients
- 5- Pour the mix in a mold
- 6- Cook for 35 minutes
- 7- Let chill




```
const nums = [1,4,3,6,7,8,9,2]
const result = []

for (let i = 0; i < nums.length; i++) {
    if (nums[i] > 5) result.push(nums[i])
}

console.log(result) // Output: [ 6, 7, 8, 9 ]
```

```
function pourIngredients() {  
  - Pour flour in a bowl  
  - Pour a couple eggs in the same bowl  
  - Pour some milk in the same bowl  
}
```

```
function mixAndTransferToMold() {  
  - Mix the ingredients  
  - Pour the mix in a mold  
}
```

```
function cookAndLetChill() {  
  - Cook for 35 minutes  
  - Let chill  
}
```

```
pourIngredients()  
mixAndTransferToMold()  
cookAndLetChill()
```

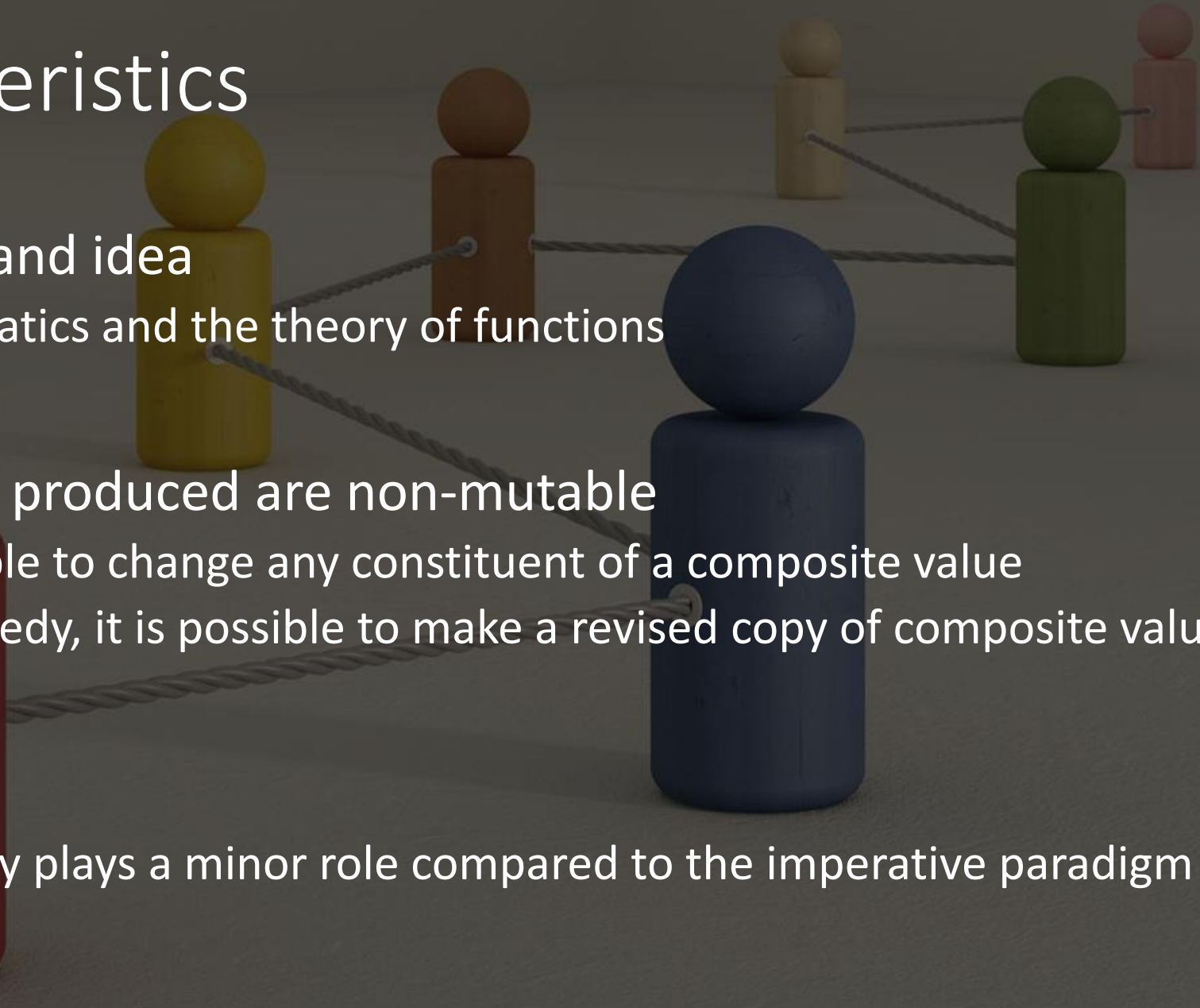


Functional Paradigm

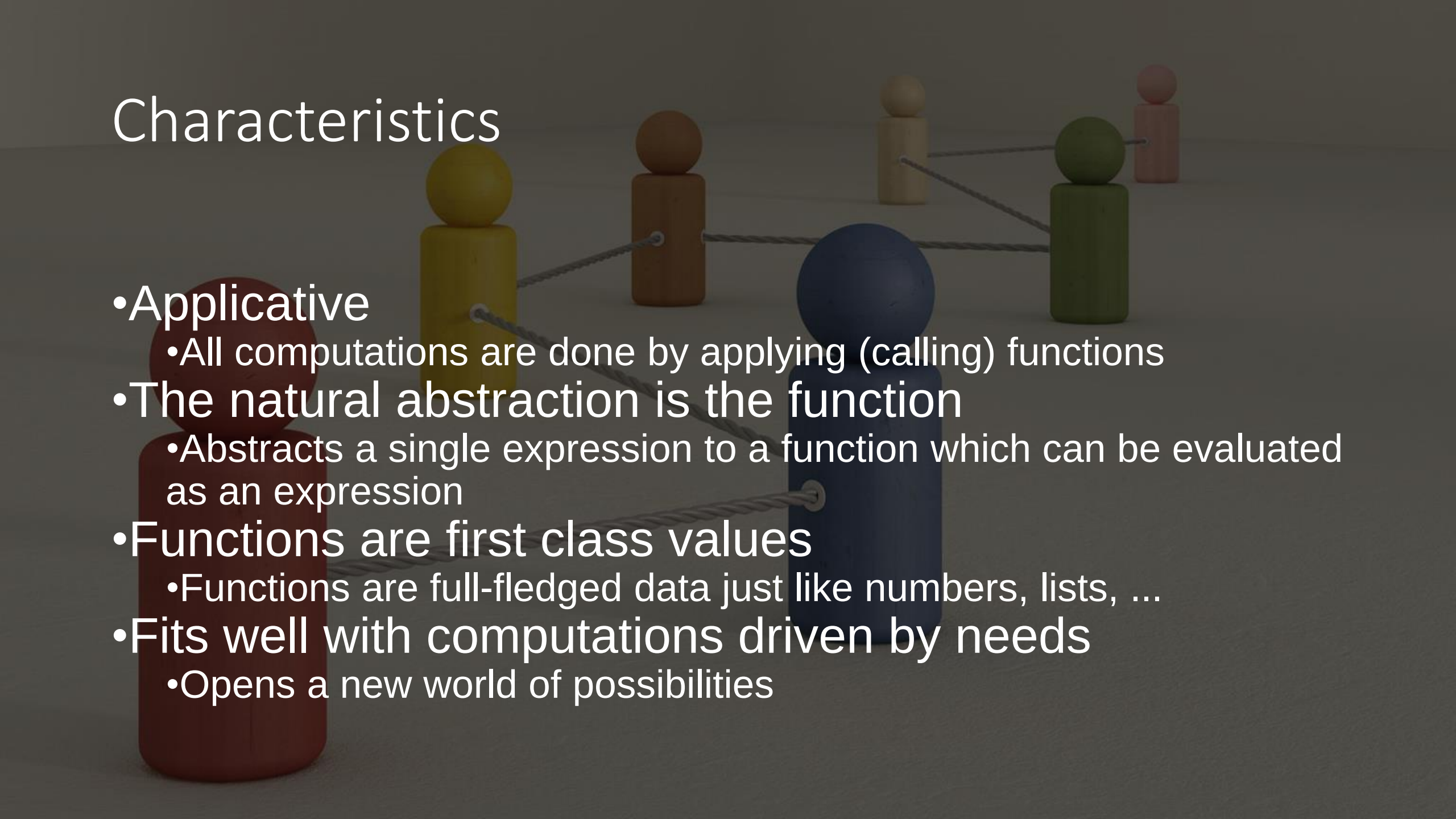
“Evaluate an expression and use the resulting value for something”

Characteristics

- Discipline and idea
 - Mathematics and the theory of functions
- The values produced are non-mutable
 - Impossible to change any constituent of a composite value
 - As a remedy, it is possible to make a revised copy of composite value
- Atemporal
 - Time only plays a minor role compared to the imperative paradigm



Characteristics

A background image showing a network of stylized human figures (cylinders with spheres on top) in various colors (red, yellow, brown, blue, green, pink) connected by thin grey ropes. The figures are arranged in a way that suggests a complex network or a system of interconnected nodes.

- **Applicative**
 - All computations are done by applying (calling) functions
- **The natural abstraction is the function**
 - Abstracts a single expression to a function which can be evaluated as an expression
- **Functions are first class values**
 - Functions are full-fledged data just like numbers, lists, ...
- **Fits well with computations driven by needs**
 - Opens a new world of possibilities

```
const nums = [1,4,3,6,7,8,9,2]
const result = [] // External variable

for (let i = 0; i < nums.length; i++) {
  if (nums[i] > 5) result.push(nums[i])
}

console.log(result) // Output: [ 6, 7, 8, 9 ]
```

```
const nums = [1,4,3,6,7,8,9,2]

function filterNums() {
  const result = [] // Internal variable

  for (let i = 0; i < nums.length; i++) {
    if (nums[i] > 5) result.push(nums[i])
  }

  return result
}

console.log(filterNums()) // Output: [ 6, 7, 8, 9 ]
```


Declarative Paradigm

“Answer a question via search for a solution-programmer states what to do but not how”

Characteristics

Discipline and idea

- Automatic proofs within artificial intelligence

Based on axioms, inference rules, and queries.

Program execution becomes a systematic search in a set of facts, making use of a set of inference rules

```
const nums = [1,4,3,6,7,8,9,2]
```

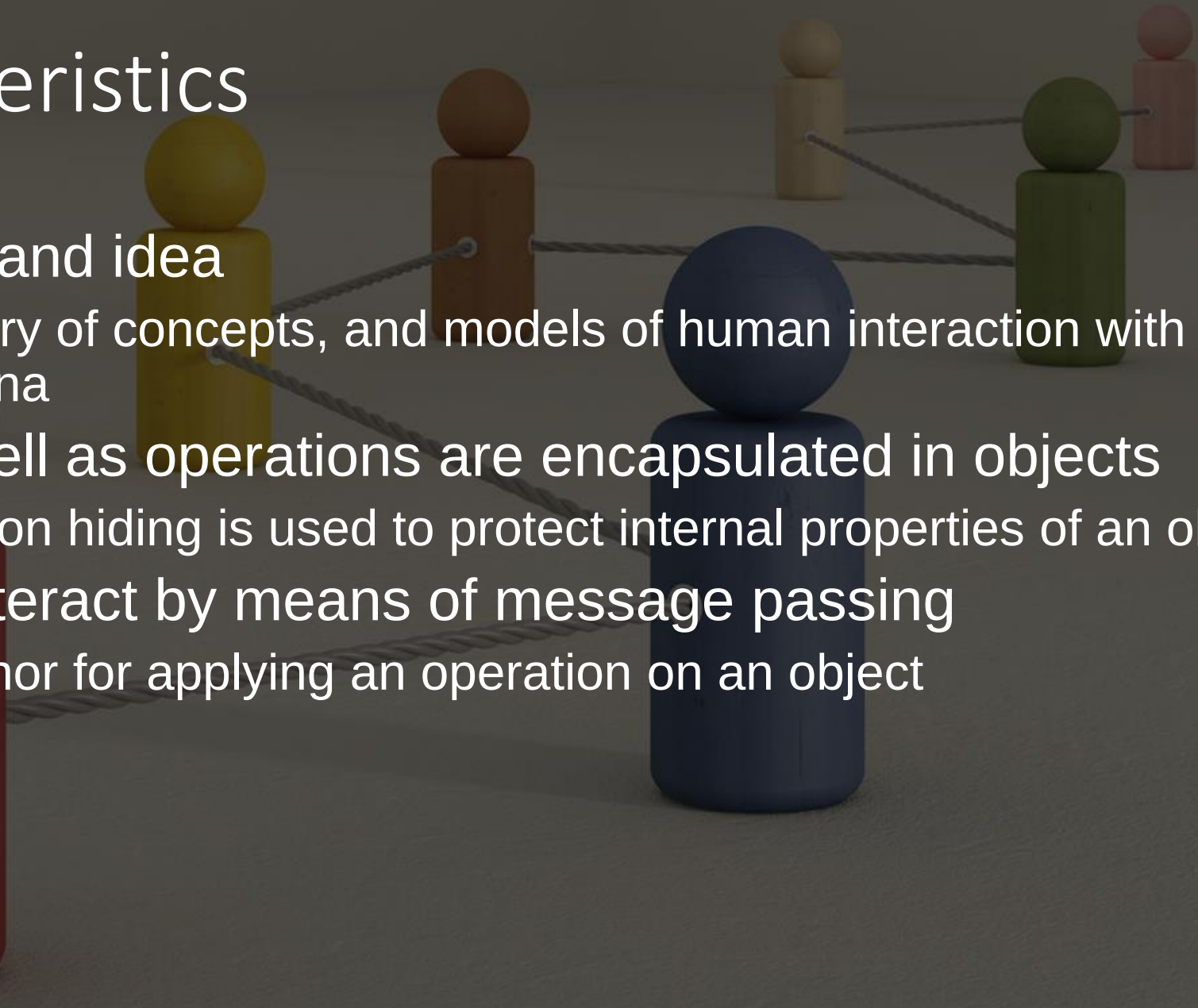
```
console.log(nums.filter(num => num > 5)) // Output: [ 6, 7, 8, 9 ]
```


Object Oriented Paradigm

“Send messages between objects to simulate the temporal evolution of a set of real world phenomena”

Characteristics

- Discipline and idea
 - The theory of concepts, and models of human interaction with real world phenomena
- Data as well as operations are encapsulated in objects
 - Information hiding is used to protect internal properties of an object
- Objects interact by means of message passing
 - A metaphor for applying an operation on an object



Characteristics

- In most object-oriented languages objects are grouped in classes
 - Objects in classes are similar enough to allow programming of the classes, as opposed to programming of the individual objects
- Classes represent concepts whereas objects represent phenomena
- Classes are organized in inheritance hierarchies
 - Provides for class extension or specialization


```
const nums = [1,4,3,6,7,8,9,2]
const result = [] // External variable

for (let i = 0; i < nums.length; i++) {
  if (nums[i] > 5) result.push(nums[i])
}

console.log(result) // Output: [ 6, 7, 8, 9 ]
```

```
// Create the two classes corresponding to each entity
class Cook {
  constructor (name) {
    this.name = name
  }

  mixAndBake() {
    - Mix the ingredients
    - Pour the mix in a mold
    - Cook for 35 minutes
  }
}

class AssistantCook {
  constructor (name) {
    this.name = name
  }

  pourIngredients() {
    - Pour flour in a bowl
    - Pour a couple eggs in the same bowl
    - Pour some milk in the same bowl
  }

  chillTheCake() {
    - Let chill
  }
}

// Instantiate an object from each class
const Frank = new Cook('Frank')
const Anthony = new AssistantCook('Anthony')

// Call the corresponding methods from each instance
Anthony.pourIngredients()
Frank.mixAndBake()
Anthony.chillTheCake()
```

Summary

Paradigms help
reason about
problems

Some languages
are multi-
paradigm

Choose what
works best for a
specific problem

Review Questions

Which paradigm ultimately underlies all others?

Which paradigm supports efficient development?

Which paradigm produces more maintainable source code?

Which paradigm reduces complexity?

Which paradigm promotes more safety and security?